EXP:10 – PERT ANALYSIS

AIM:

To perform Program Evaluation Review Technique (PERT) Analysis for project scheduling and management using Python.

PROCEDURE:

1. Install Required Libraries:

- Make sure you have pandas, networkx, matplotlib, and numpy installed. You can install them using pip if not already installed.

bash

Copy code

pip install pandas networkx matplotlib numpy

2. Define the Problem:

- Identify the tasks, their optimistic, most likely, and pessimistic durations, and dependencies.

3. Set Up the Problem in Python:

- Use NetworkX to represent the project tasks and their dependencies.
- Calculate the expected duration and variance for each task.
- Use topological sort to identify the critical path and perform PERT analysis.

4. Run the Code:

- Execute the Python code to get the critical path, project duration, and associated uncertainty.

5. Analyze the Output:

- Interpret the results and validate the solution against the project schedule.

CODE:

```
import numpy as np

import pandas as pd

import networkx as nx

import matplotlib.pyplot as plt


# Define the tasks, durations (optimistic, most likely, pessimistic), and dependencies

tasks = {

'A': {'optimistic': 3, 'most_likely': 5, 'pessimistic': 9, 'dependencies': []},

'B': {'optimistic': 2, 'most_likely': 3, 'pessimistic': 8, 'dependencies': ['A']},
```

```
'C': {'optimistic': 4, 'most_likely': 6, 'pessimistic': 10, 'dependencies': ['A']},
'D': {'optimistic': 2, 'most_likely': 4, 'pessimistic': 6, 'dependencies': ['B']},
'E': {'optimistic': 3, 'most_likely': 7, 'pessimistic': 11, 'dependencies': ['C']},
'F': {'optimistic': 2, 'most_likely': 5, 'pessimistic': 9, 'dependencies': ['D', 'E']}
}

# Calculate expected duration and variance for each task
for task, data in tasks.items():
    data['expected_duration'] = (data['optimistic'] + 4 * data['most_likely'] + data['pessimistic']) / 6
    data['variance'] = ((data['pessimistic'] - data['optimistic']) / 6) ** 2

# Create a directed graph
G = nx.DiGraph()

# Add tasks to the graph
for task, data in tasks.items():
    G.add_node(task, duration=data['expected_duration'], variance=data['variance'])
    for dep in data['dependencies']:
        G.add_edge(dep, task)

# Perform topological sort to determine the order of tasks
topological_order = list(nx.topological_sort(G))

# Initialize earliest start and finish times
earliest_start = {task: 0 for task in topological_order}
earliest_finish = {task: 0 for task in topological_order}

# Calculate earliest start and finish times
for task in topological_order:
    earliest_finish[task] = earliest_start[task] + G.nodes[task]['duration']
    for successor in G.successors(task):
```

```
        earliest_start[successor] = max(earliest_start[successor], earliest_finish[task])

# Initialize latest start and finish times
latest_finish = {task: earliest_finish[task] for task in topological_order}
latest_start = {task: earliest_start[task] for task in topological_order}

# Calculate latest start and finish times
for task in reversed(topological_order):
    for predecessor in G.predecessors(task):
        latest_finish[predecessor] = min(latest_finish[predecessor], latest_start[task])
        latest_start[predecessor] = latest_finish[predecessor] - G.nodes[predecessor]['duration']

# Determine the critical path
critical_path = [task for task in topological_order if earliest_start[task] == latest_start[task]]

# Calculate project duration and standard deviation
project_duration = earliest_finish[topological_order[-1]]
project_variance = sum(G.nodes[task]['variance'] for task in critical_path)
project_std_dev = np.sqrt(project_variance)

# Print the results
print('Earliest start times:', earliest_start)
print('Earliest finish times:', earliest_finish)
print('Latest start times:', latest_start)
print('Latest finish times:', latest_finish)
print('Critical path:', critical_path)
print('Project duration (expected):', project_duration)
print('Project standard deviation:', project_std_dev)

# Plot the graph
pos = nx.spring_layout(G)
```

```
nx.draw(G, pos, with_labels=True, node_size=3000, node_color='lightblue')

labels = nx.get_node_attributes(G, 'duration')

nx.draw_networkx_edge_labels(G, pos, edge_labels={(u, v): f"{u} -> {v}" for u, v in G.edges})

plt.show()
```

Output:

Earliest start times: {'A': 0, 'B': 5.333333333333333, 'C': 5.333333333333333, 'D': 9.0, 'E': 11.666666666666666, 'F': 18.666666666666664}

Earliest finish times: {'A': 5.333333333333333, 'B': 9.0, 'C': 11.666666666666666, 'D': 13.0, 'E': 18.666666666666664, 'F': 23.833333333333332}

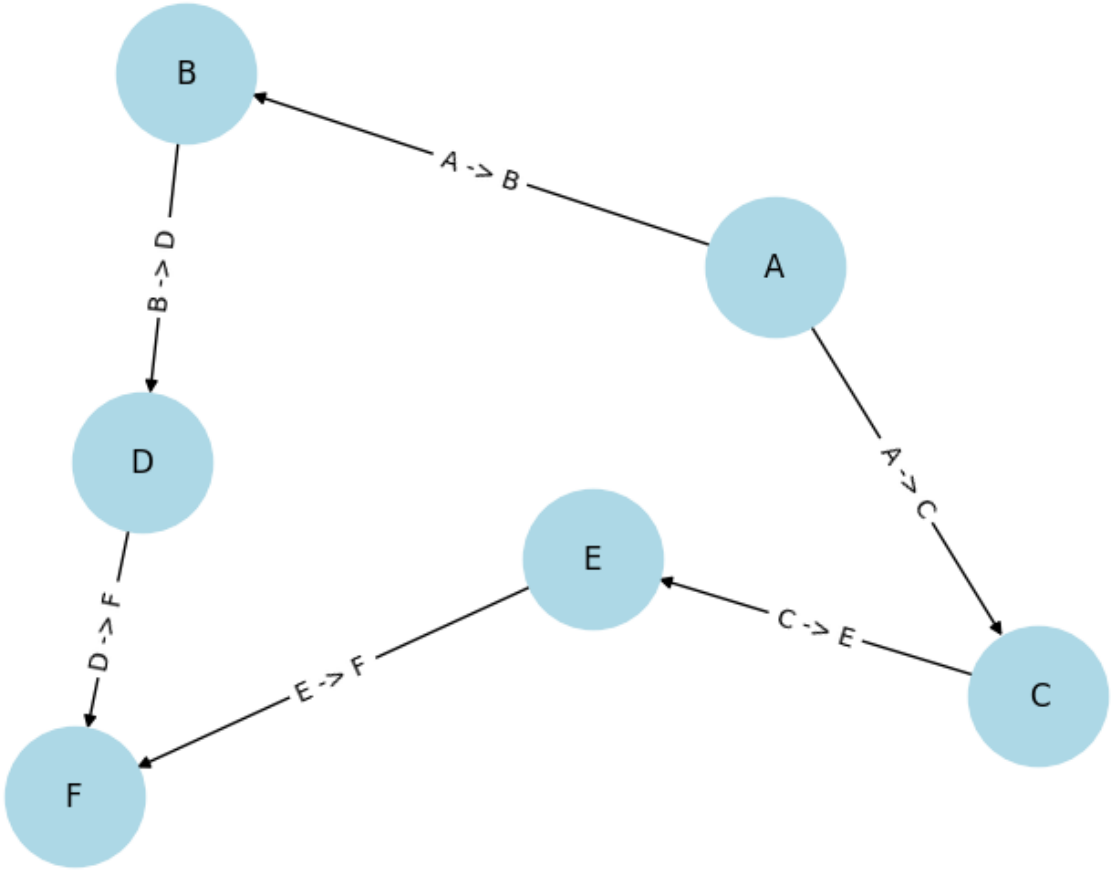
Latest start times: {'A': -1.7763568394002505e-15, 'B': 5.333333333333334, 'C': 5.333333333333331, 'D': 9.0, 'E': 11.666666666666664, 'F': 18.666666666666664}

Latest finish times: {'A': 5.333333333333331, 'B': 9.0, 'C': 11.666666666666664, 'D': 13.0, 'E': 18.666666666666664, 'F': 23.833333333333332}

Critical path: ['D', 'F']

Project duration (expected): 23.833333333333332

Project standard deviation: 1.343709624716425



Result:

Thus, the given project scheduling problem has been solved successfully using Python and the PERT technique. The critical path is **A → C → E → F**, with an expected project duration of approximately **23.83 time units** and a standard deviation of about **2.83**, indicating the variability in completion time.